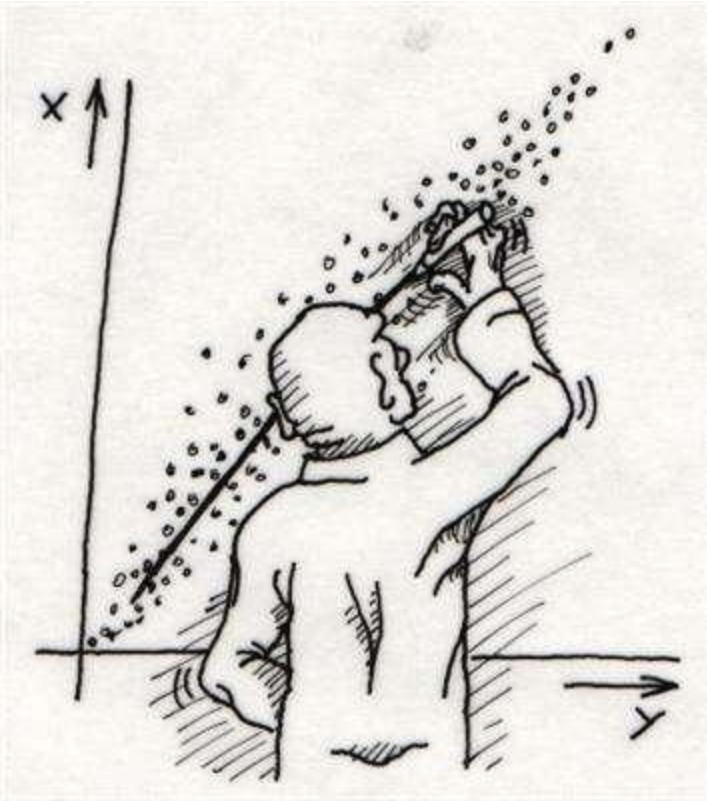
Fitting: Concepts and recipes



Review: The Hough transform

- What are the steps of a Hough transform?
- What line parametrization is best for the Hough transform?
- How does bin size affect the performance of the Hough transform?
- What is the complexity of the Hough transform as a function of the number of parameters in the model?
- How can we use gradient direction to modify the Hough transform?
- How do we find circles with the Hough transform?
- What about general shapes?

Today: A “grab-bag” of techniques

- If we know which points belong to the line, how do we find the “optimal” line parameters?
  - Least squares
  - Probabilistic fitting
- What if there are outliers?
  - Robust fitting, RANSAC
- What if there are many lines?
  - Incremental fitting, K-lines, EM
- What if we’re not even sure it’s a line?
  - Model selection
- Our main case study remains line fitting, but most of the concepts are very generic and widely applicable

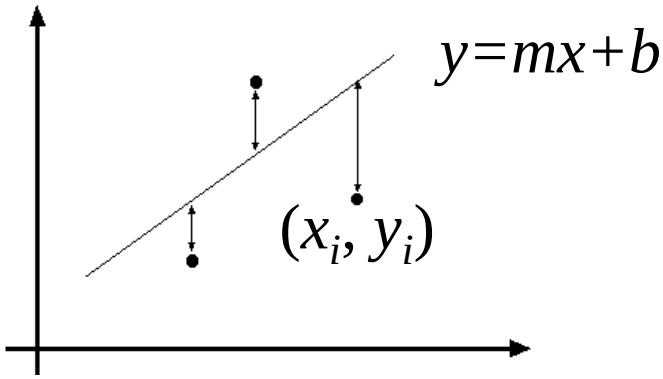
Least squares line fitting

Data: $(x_1, y_1), \ldots, (x_n, y_n)$

Line equation: $y_i = m x_i + b$

Find (m, b) to minimize

$$E = \sum_{i=1}^{n} (y_i - m x_i - b)^2$$

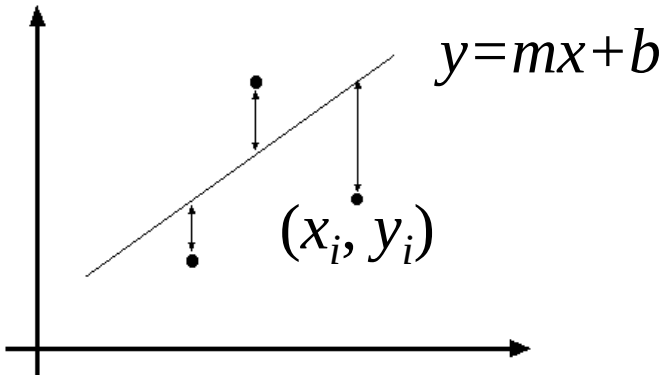


Least squares line fitting

Data: $(x_1, y_1), \ldots, (x_n, y_n)$

Line equation: $y_i = m x_i + b$

Find (m, b) to minimize



$$\boxed{E = \sum_{i=1}^{n} (y_i - m x_i - b)^2}$$

$$E = \sum_{i=1}^{n} \left(y_i - \begin{bmatrix} x_i & 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} \right)^2$$

$$\frac{dE}{dB} = 2X^T XB - 2X^T Y = 0$$

$$\boxed{X^T XB = X^T Y}$$

Normal equations: least squares solution to $XB=Y$

Problem with “vertical” least squares

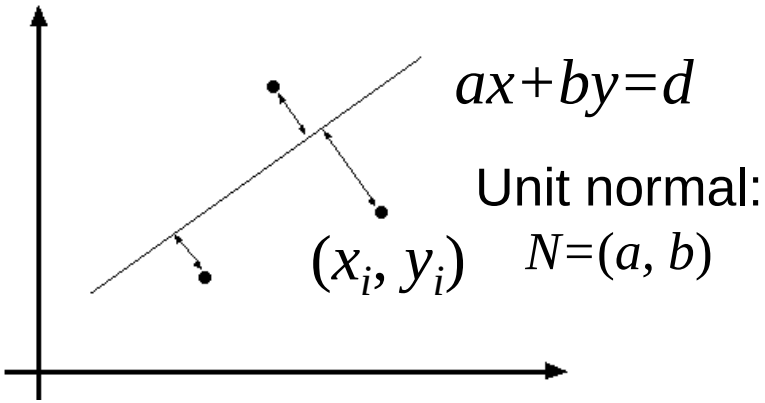
- Not rotation-invariant
- Fails completely for vertical lines

Total least squares

Distance between point (x_n, y_n) and line $ax+by=d$ $(a^2+b^2=1)$: $|ax + by - d|$

Find (a, b, d) to minimize the sum of squared perpendicular distances

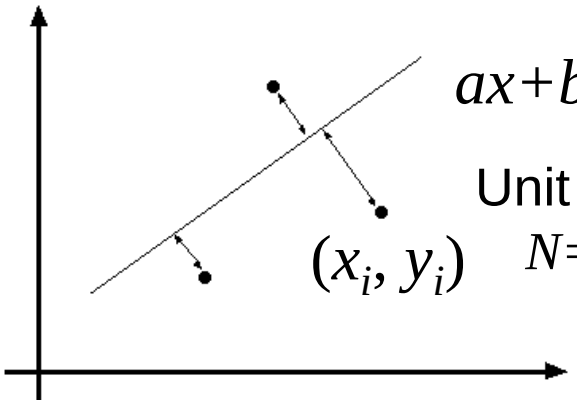
$$\boxed{E = \sum_{i=1}^{n} (ax_i + by_i - d)^2}$$



Total least squares

Distance between point (x_n, y_n) and line $ax+by=d$ $(a^2+b^2=1)$: $|ax + by - d|$

Find (a, b, d) to minimize the sum of squared perpendicular distances



Unit normal: $N=(a, b)$

$$\boxed{E = \sum_{i=1}^{n} (ax_i + by_i - d)^2}$$

$$\frac{\partial E}{\partial d} = \sum_{i=1}^{n} -2(ax_i + by_i - d) = 0$$

$$d = \frac{a}{n}\sum_{i=1}^{n} x_i + \frac{b}{n}\sum_{i=1}^{n} x_i = a\bar{x} + b\bar{y}$$

$$E = \sum_{i=1}^{n} (a(x_i - \bar{x}) + b(y_i - \bar{y}))^2 = \left\| \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} \\ \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} \right\|^2 = (UN)^T (UN)$$

$$\frac{dE}{dN} = 2(U^T U)N = 0$$

Solution to $(U^T U)N = 0$, subject to $\|N\|^2 = 1$: eigenvector of $U^T U$ associated with the smallest eigenvalue (least squares solution to *homogeneous linear system* $UN = 0$)

Total least squares

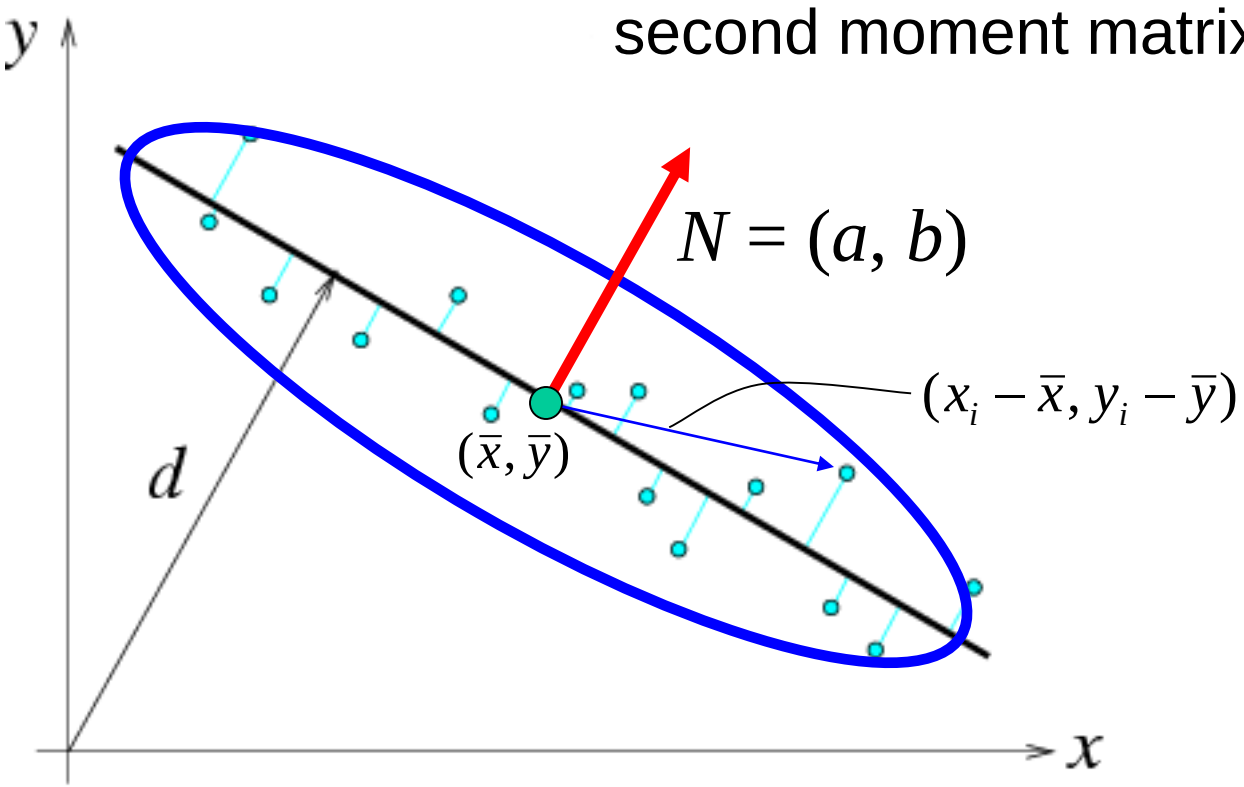
$$U = \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} \\ \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} \end{bmatrix} \qquad U^T U = \begin{bmatrix} \sum_{i=1}^{n} (x_i - \bar{x})^2 & \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) \\ \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) & \sum_{i=1}^{n} (y_i - \bar{y})^2 \end{bmatrix}$$

second moment matrix

Total least squares

$$U = \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} \\ \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} \end{bmatrix} \qquad U^T U = \begin{bmatrix} \sum_{i=1}^{n} (x_i - \bar{x})^2 & \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) \\ \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) & \sum_{i=1}^{n} (y_i - \bar{y})^2 \end{bmatrix}$$

second moment matrix



Least squares as likelihood maximization

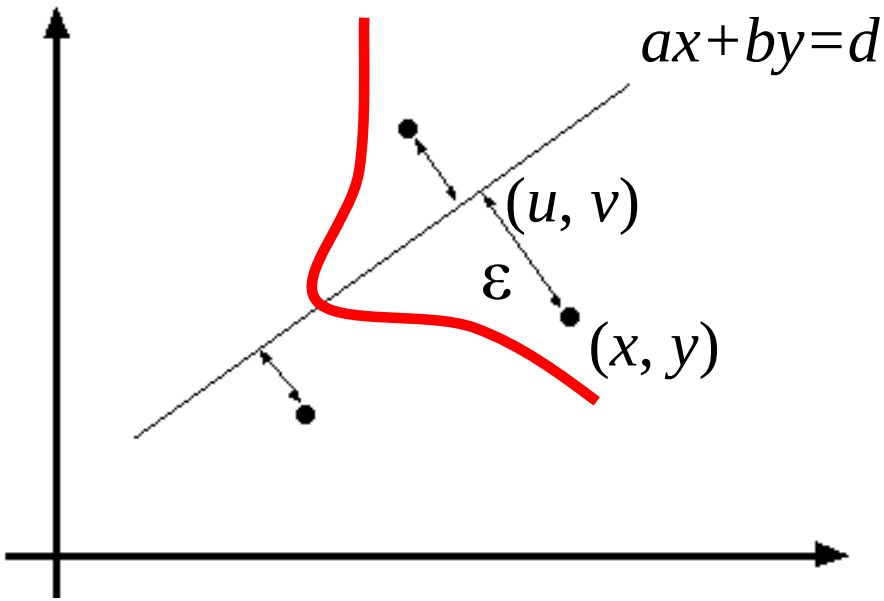
- **Generative model**: line points are corrupted by Gaussian noise in the direction perpendicular to the line

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} u \\ v \end{pmatrix} + \varepsilon \begin{pmatrix} a \\ b \end{pmatrix}$$

point on the line

noise: zero-mean Gaussian with std. dev. σ

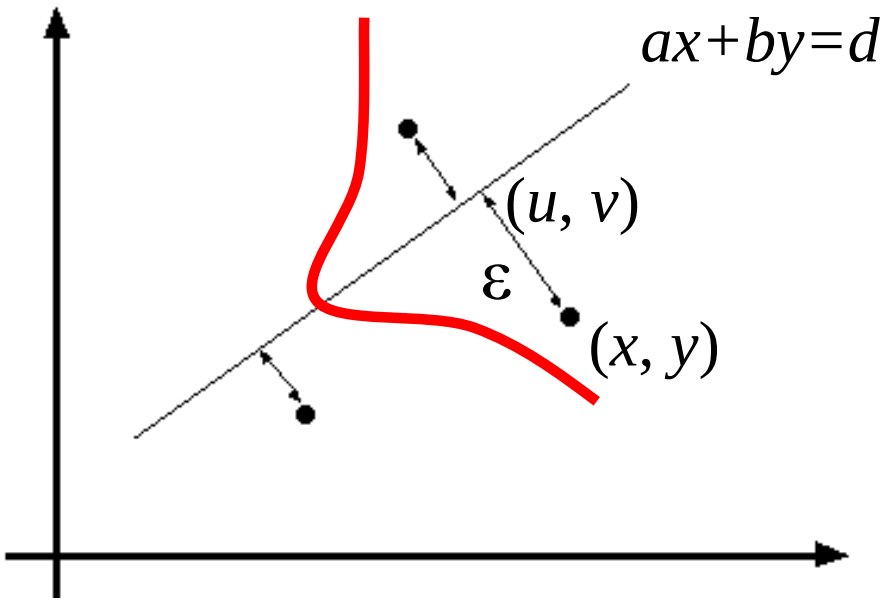
normal direction



Least squares as likelihood maximization

- **Generative model**: line points are corrupted by Gaussian noise in the direction perpendicular to the line

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} u \\ v \end{pmatrix} + \varepsilon \begin{pmatrix} a \\ b \end{pmatrix}$$



Likelihood of points given line parameters (*a*, *b*, *d*):

$$P(x_1,\ldots,x_n \mid a,b,d) = \prod_{i=1}^{n} P(x_i \mid a,b,d) \propto \prod_{i=1}^{n} \exp\left(-\frac{(ax_i + by_i - d)^2}{2\sigma^2}\right)$$

Log-likelihood: $L(x_1,\ldots,x_n \mid a,b,d) = -\frac{1}{2\sigma^2}\sum_{i=1}^{n}(ax_i + by_i - d)^2$

Probabilistic fitting: General concepts

- Likelihood:

$$L(\theta) = P(x_1, \ldots, x_n \mid \theta) = \prod_{i=1}^{n} P(x_i \mid \theta)$$

Probabilistic fitting: General concepts

- Likelihood:

$$L(\theta) = P(x_1, \ldots, x_n \mid \theta) = \prod_{i=1}^{n} P(x_i \mid \theta)$$

- Log-likelihood:

$$\log L(\theta) = \log P(x_1, \ldots, x_n \mid \theta) = \sum_{i=1}^{n} \log P(x_i \mid \theta)$$

Probabilistic fitting: General concepts

- Likelihood:

$$L(\theta) = P(x_1, \ldots, x_n \mid \theta) = \prod_{i=1}^{n} P(x_i \mid \theta)$$

- Log-likelihood:

$$\log L(\theta) = \log P(x_1, \ldots, x_n \mid \theta) = \sum_{i=1}^{n} \log P(x_i \mid \theta)$$

- **Maximum likelihood** estimation:

$$\hat{\theta} = \arg\max_{\theta} L(\theta)$$

Probabilistic fitting: General concepts

- Likelihood:

$$L(\theta) = P(x_1, \ldots, x_n \mid \theta) = \prod_{i=1}^{n} P(x_i \mid \theta)$$

- Log-likelihood:

$$\log L(\theta) = \log P(x_1, \ldots, x_n \mid \theta) = \sum_{i=1}^{n} \log P(x_i \mid \theta)$$

- **Maximum likelihood** estimation:

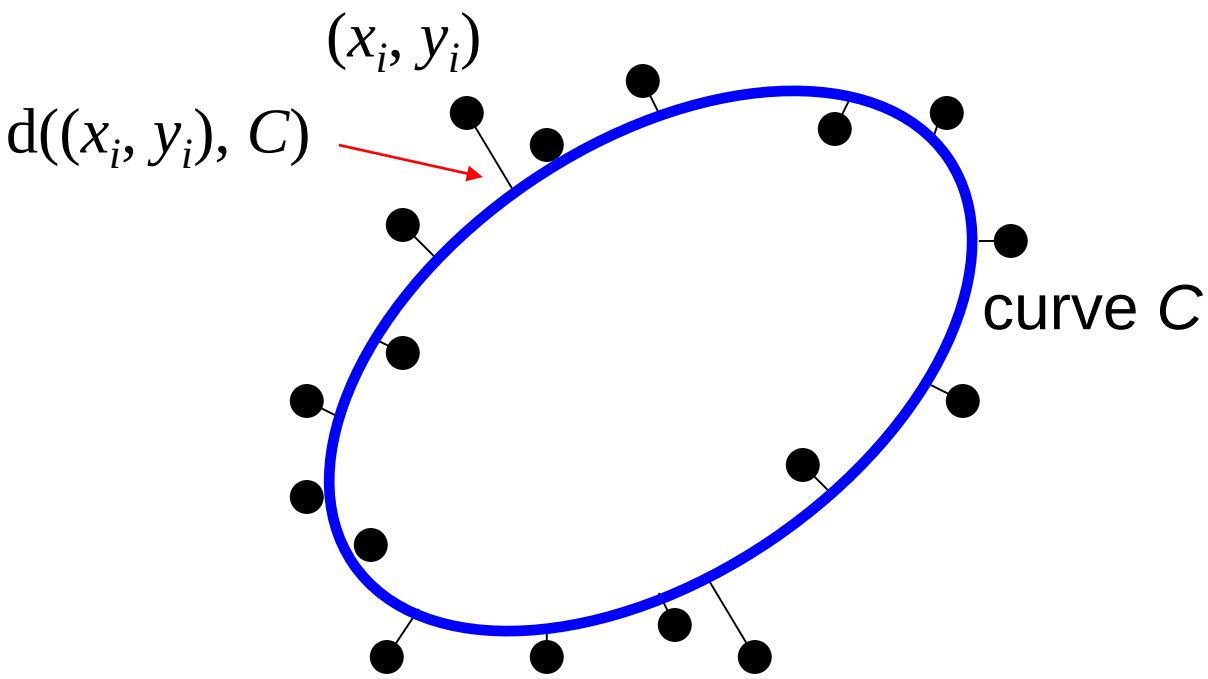
$$\hat{\theta} = \arg\max_{\theta} L(\theta)$$

- **Maximum a posteriori** (MAP) estimation:

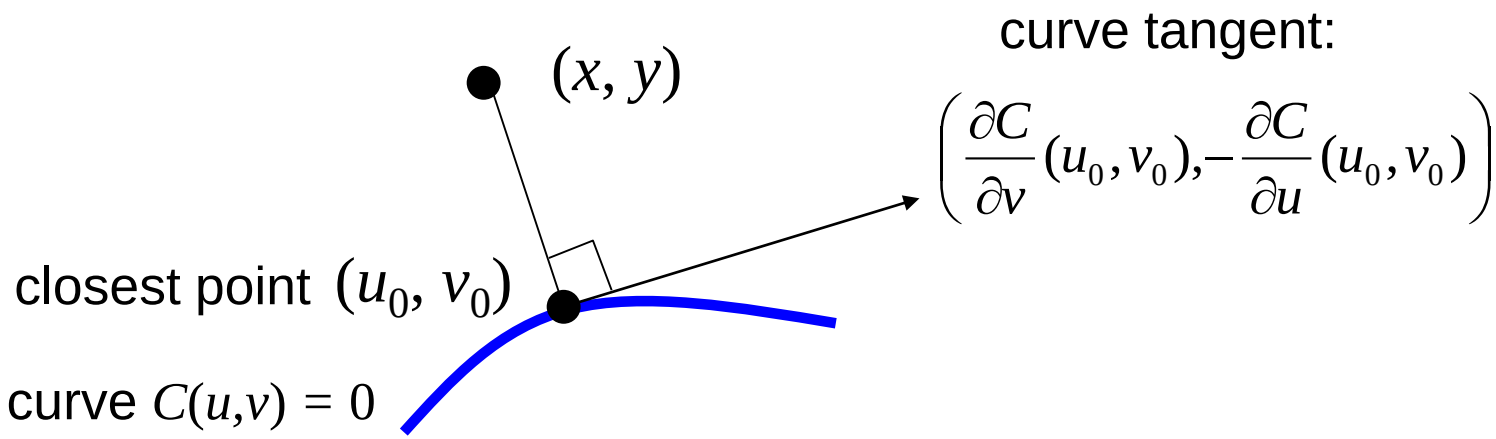
$$\hat{\theta} = \arg\max_{\theta} P(\theta \mid x_1, \ldots, x_n) \propto \arg\max_{\theta} L(\theta) \underbrace{P(\theta)}_{\text{prior}}$$

Least squares for general curves

- We would like to minimize the sum of squared *geometric distances* between the data points and the curve



Calculating geometric distance



The curve tangent must be orthogonal to the vector connecting (x, y) with the closest point on the curve, (u_0, v_0):

$$\begin{cases} \frac{\partial C}{\partial v}(u_0, v_0)[x - u_0] - \frac{\partial C}{\partial v}(u_0, v_0)[y - v_0] = 0 \\ C(u_0, v_0) = 0 \end{cases}$$

Must solve system of equations for (u_0, v_0)

Least squares for conics

- Equation of a general conic:

  $C(\mathbf{a}, \mathbf{x}) = \mathbf{a} \cdot \mathbf{x} = ax^2 + bxy + cy^2 + dx + ey + f = 0,$

  $\mathbf{a} = [a, b, c, d, e, f],$

  $\mathbf{x} = [x^2, xy, y^2, x, y, 1]$

- Minimizing the geometric distance is non-linear even for a conic
- *Algebraic distance:* $C(\mathbf{a}, \mathbf{x})$
- Algebraic distance minimization by linear least squares:

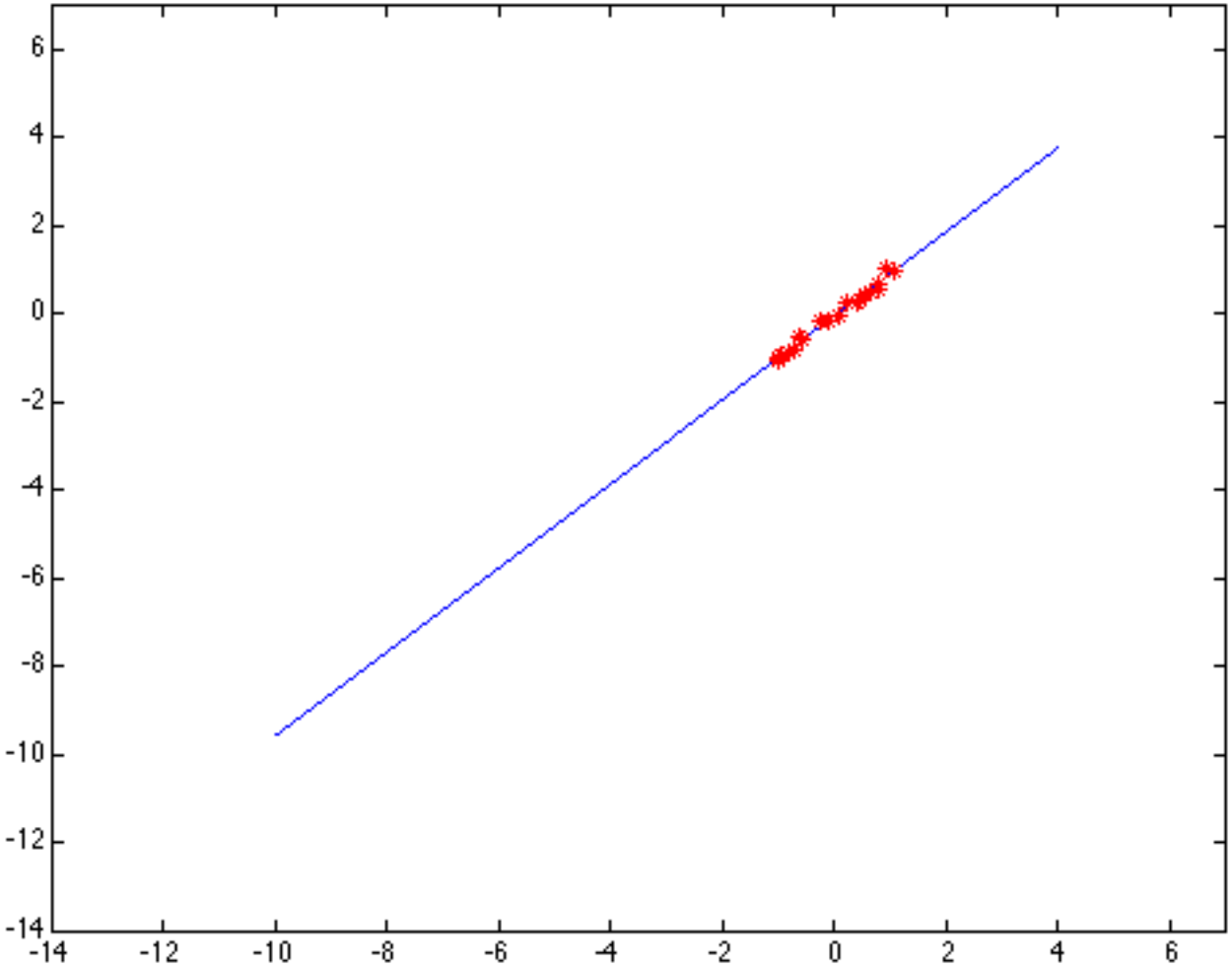
$$\begin{bmatrix} x_1^2 & x_1 y_1 & y_1^2 & x_1 & y_1 & 1 \\ x_2^2 & x_2 y_2 & y_2^2 & x_2 & y_2 & 1 \\ \vdots & & & & \ddots & \vdots \\ x_n^2 & x_n y_n & y_n^2 & x_n & y_n & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \end{bmatrix} = 0$$

Least squares for conics

- Least squares system: D**a** = 0
- Need constraint on **a** to prevent trivial solution
- Discriminant: $b^2 - 4ac$
  - Negative: ellipse
  - Zero: parabola
  - Positive: hyperbola
- Minimizing squared algebraic distance subject to constraints leads to a generalized eigenvalue problem
  - Many variations possible
- For more information:
  - A. Fitzgibbon, M. Pilu, and R. Fisher, *Direct least-squares fitting of ellipses,* EEE Transactions on Pattern Analysis and Machine Intelligence, 21(5), 476--480, May 1999

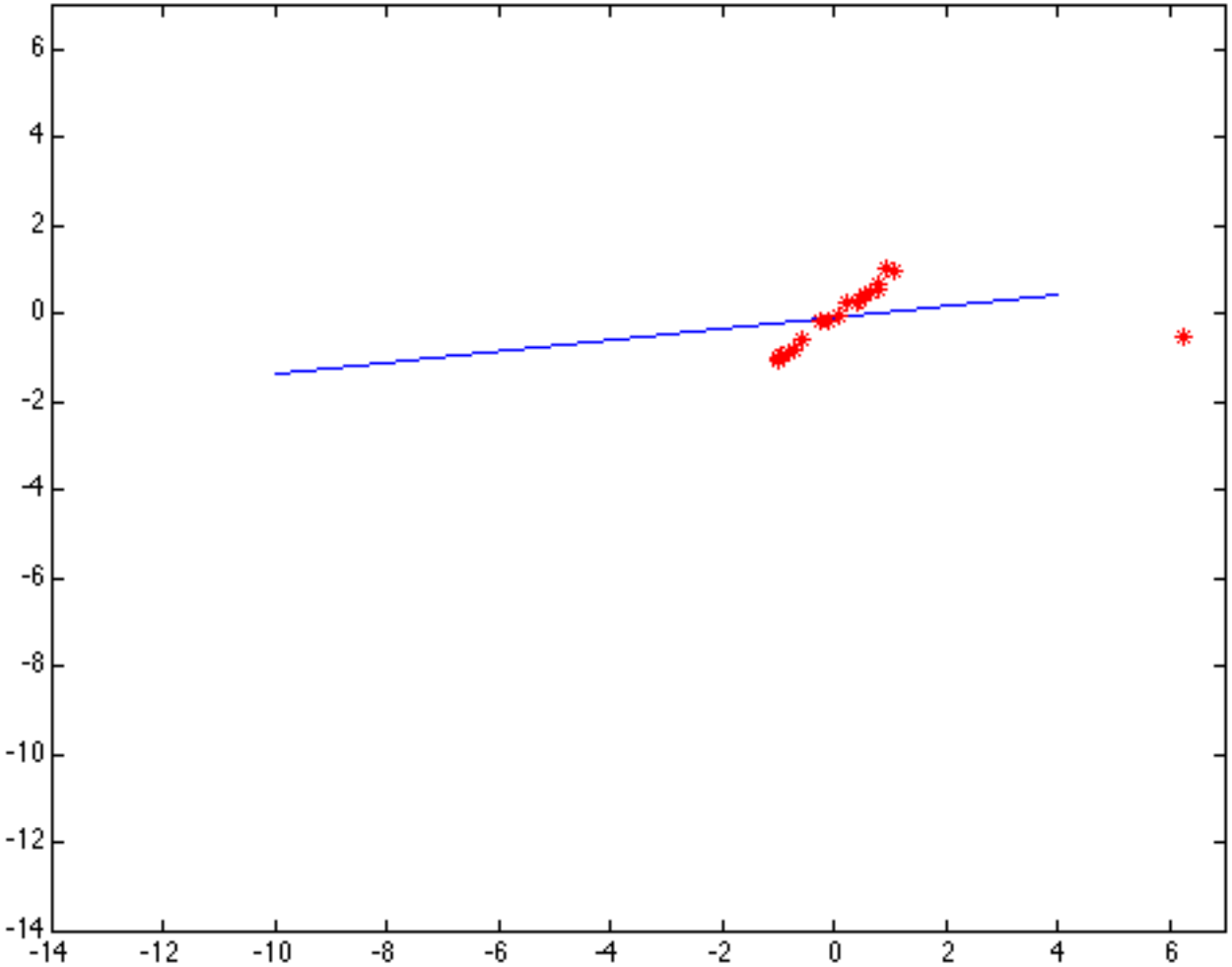
Least squares: Robustness to noise

Least squares fit to the red points:



Least squares: Robustness to noise

Least squares fit with an outlier:

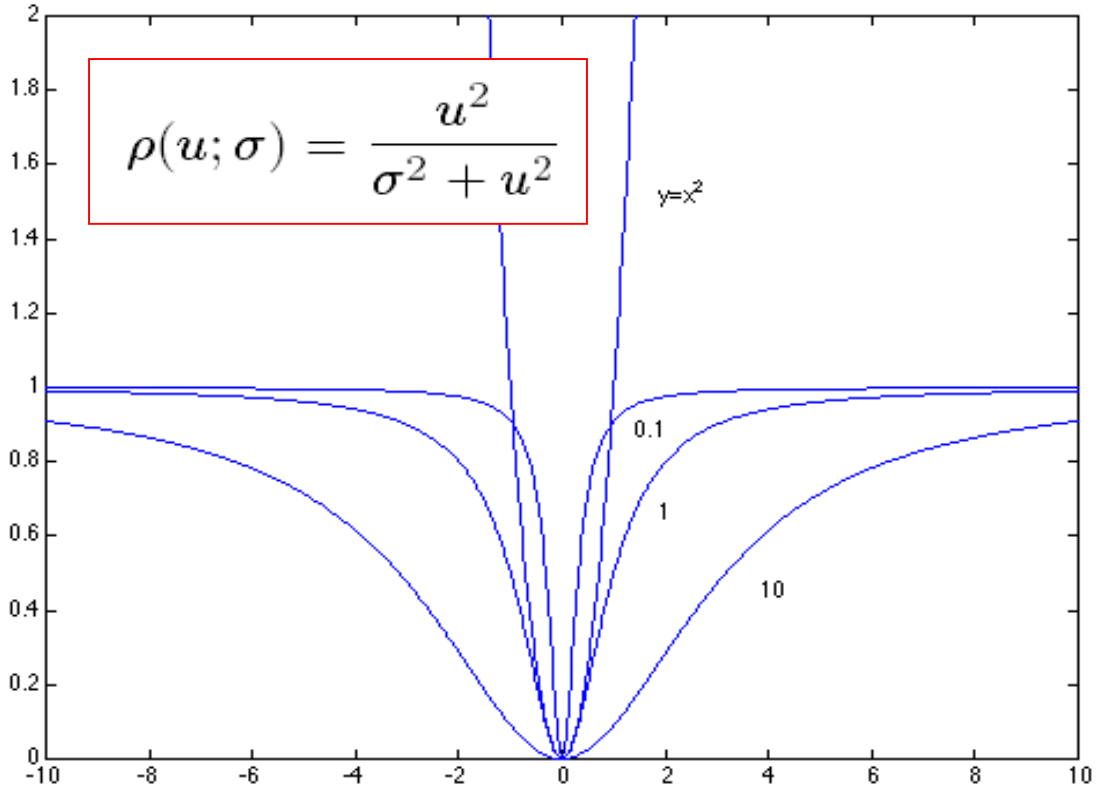


Problem: squared error heavily penalizes outliers

Robust estimators

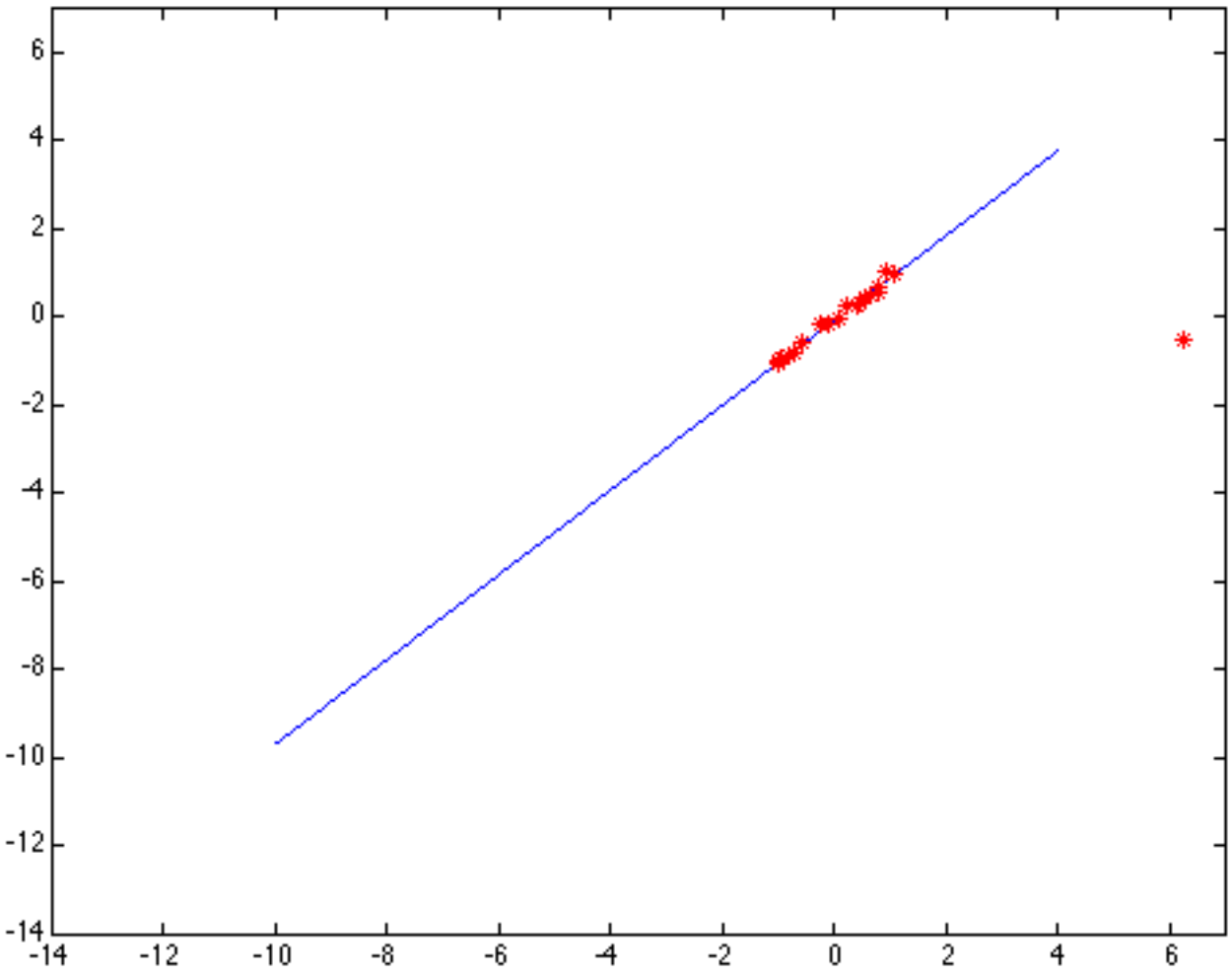
- General approach: minimize $\sum_i \rho\left(r_i\left(x_i, \theta\right); \sigma\right)$

  $r_i(x_i, \theta)$ – residual of ith point w.r.t. model parameters θ
  ρ – robust function with scale parameter σ



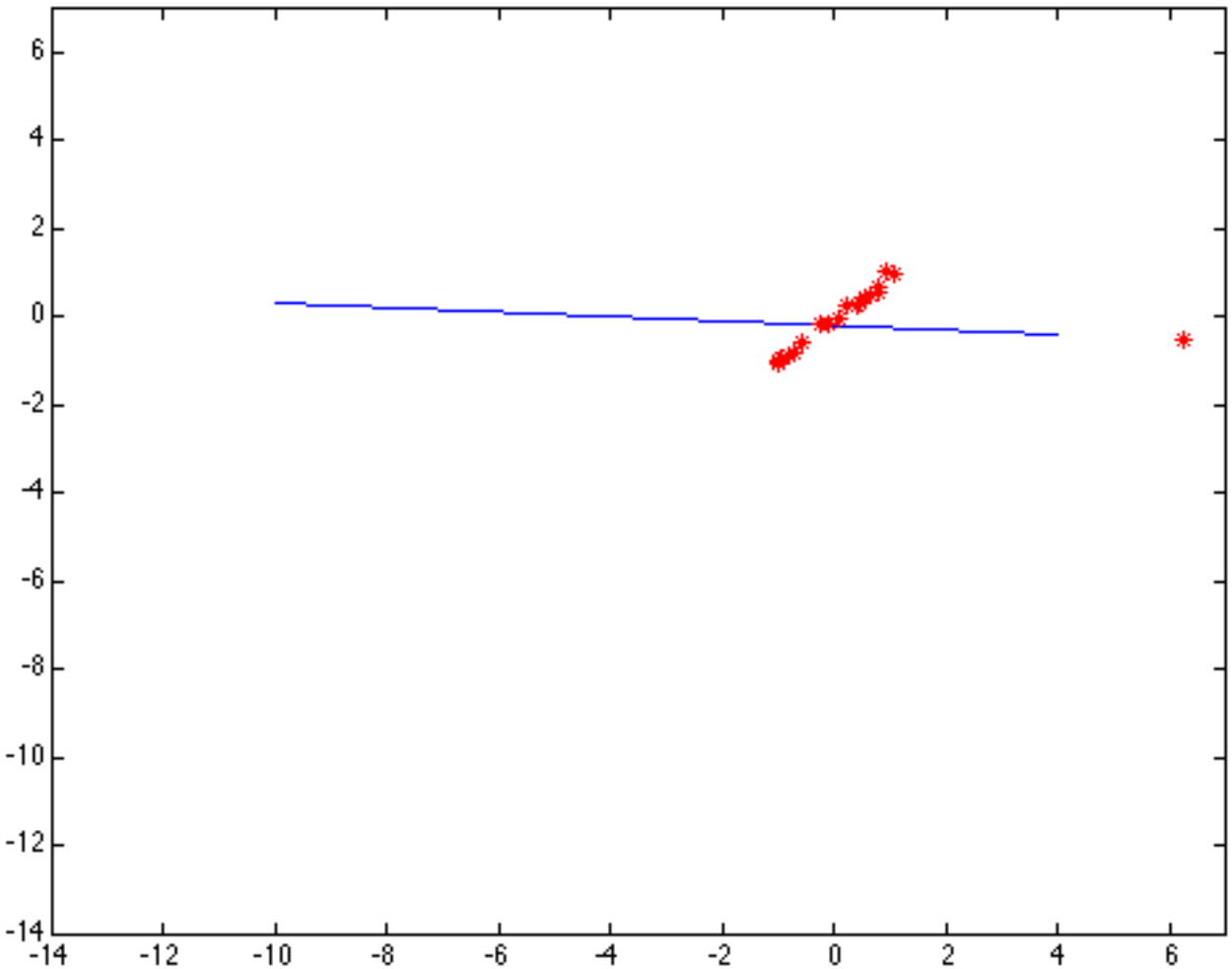
The robust function ρ behaves like squared distance for small values of the residual *u* but saturates for larger values of *u*

Choosing the scale: Just right



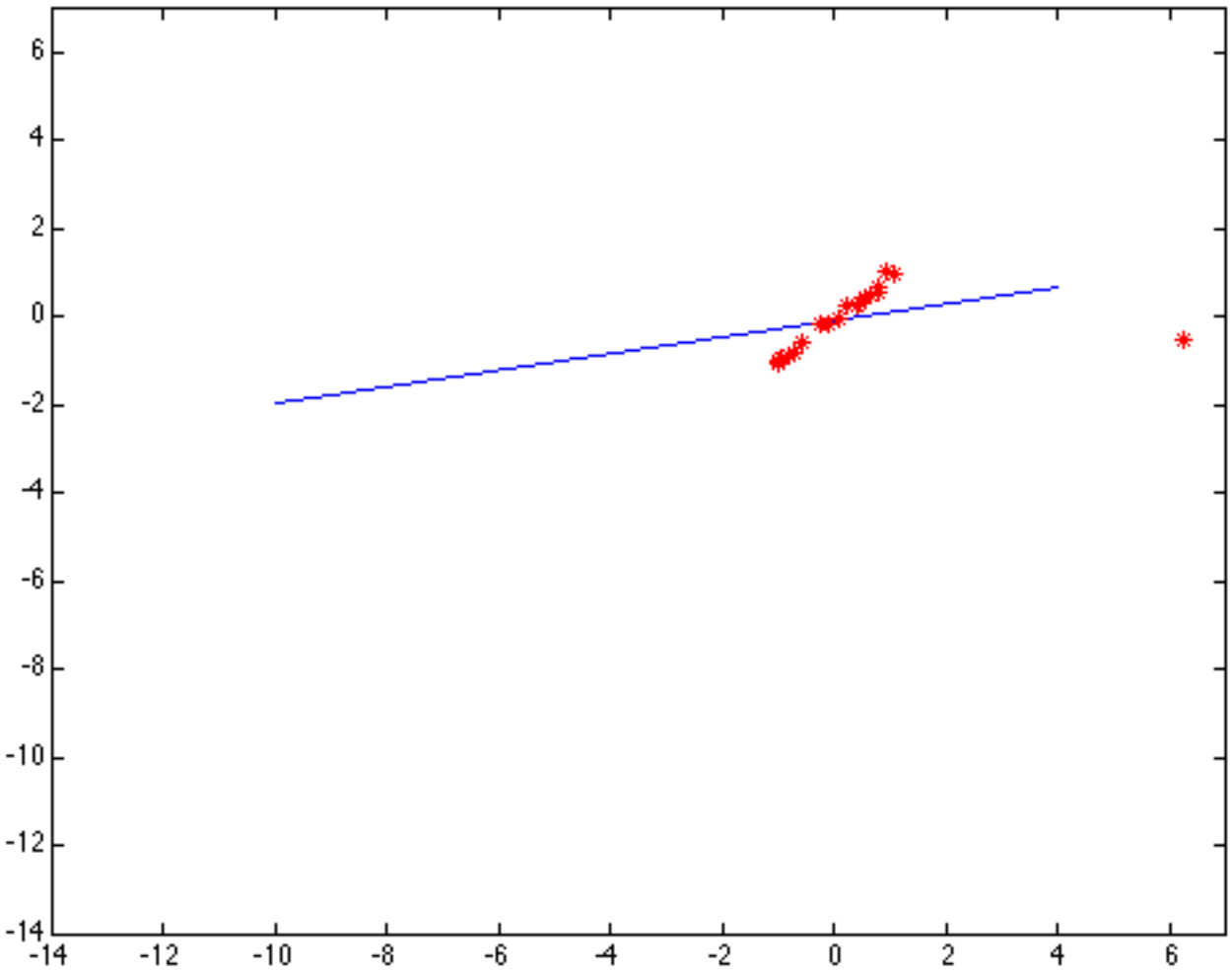
The effect of the outlier is eliminated

Choosing the scale: Too small



The error value is almost the same for every point and the fit is very poor

Choosing the scale: Too large



Behaves much the same as least squares

Robust estimation: Notes

- Robust fitting is a nonlinear optimization problem that must be solved iteratively
- Least squares solution can be used for initialization
- Adaptive choice of scale:
  "magic number" times median residual

$$\sigma^{(n)} = 1.4826 \text{ median}_i \; |r_i^{(n)}(x_i; \theta^{(n-1)})|$$

RANSAC

- Robust fitting can deal with a few outliers – what if we have very many?
- Random sample consensus (RANSAC): Very general framework for model fitting in the presence of outliers
- Outline
  - Choose a small subset uniformly at random
  - Fit a model to that subset
  - Find all remaining points that are “close” to the model and reject the rest as outliers
  - Do this many times and choose the best model

M. A. Fischler, R. C. Bolles. Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. Comm. of the ACM, Vol 24, pp 381-395, 1981.

RANSAC for line fitting

Repeat ***N*** times:

- Draw ***s*** points uniformly at random
- Fit line to these ***s*** points
- Find inliers to this line among the remaining points (i.e., points whose distance from the line is less than ***t***)
- If there are ***d*** or more inliers, accept the line and refit using all inliers

Choosing the parameters

- Initial number of points *s*
  - Typically minimum number needed to fit the model
- Distance threshold *t*
  - Choose *t* so probability for inlier is *p* (e.g. 0.95)
  - Zero-mean Gaussian noise with std. dev. σ: $t^2=3.84\sigma^2$
- Number of samples *N*
  - Choose *N* so that, with probability *p*, at least one random sample is free from outliers (e.g. *p*=0.99) (outlier ratio: *e*)

Source: M. Pollefeys

Choosing the parameters

- Initial number of points *s*
  - Typically minimum number needed to fit the model
- Distance threshold *t*
  - Choose *t* so probability for inlier is *p* (e.g. 0.95)
  - Zero-mean Gaussian noise with std. dev. σ: $t^2=3.84\sigma^2$
- Number of samples *N*
  - Choose *N* so that, with probability *p*, at least one random sample is free from outliers (e.g. *p*=0.99) (outlier ratio: *e*)

$$\boxed{\left(1-(1-e)^s\right)^N = 1-p}$$

$$N = \log(1-p)/\log\left(1-(1-e)^s\right)$$

	proportion of outliers *e*						
s	5%	10%	20%	25%	30%	40%	50%
2	2	3	5	6	7	11	17
3	3	4	7	9	11	19	35
4	3	5	9	13	17	34	72
5	4	6	12	17	26	57	146
6	4	7	16	24	37	97	293
7	4	8	20	33	54	163	588
8	5	9	26	44	78	272	1177

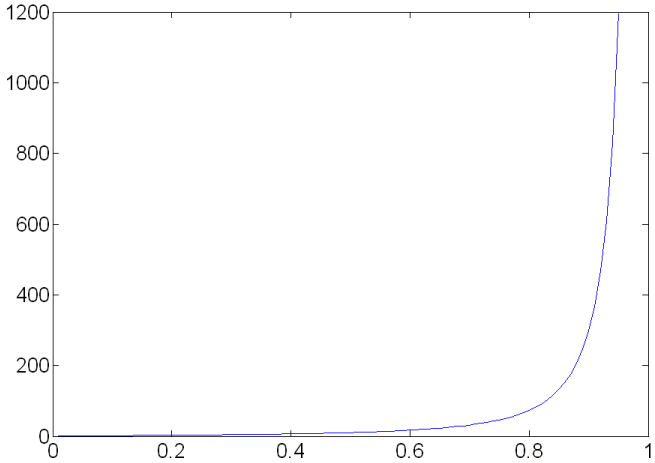
Source: M. Pollefeys

Choosing the parameters

- Initial number of points *s*
  - Typically minimum number needed to fit the model
- Distance threshold *t*
  - Choose *t* so probability for inlier is *p* (e.g. 0.95)
  - Zero-mean Gaussian noise with std. dev. σ: $t^2=3.84\sigma^2$
- Number of samples *N*
  - Choose *N* so that, with probability *p*, at least one random sample is free from outliers (e.g. *p*=0.99) (outlier ratio: *e*)

$$\boxed{\left(1-(1-e)^s\right)^N = 1-p}$$

$$N = \log(1-p)/\log\left(1-(1-e)^s\right)$$



Source: M. Pollefeys

Choosing the parameters

- Initial number of points *s*
  - Typically minimum number needed to fit the model
- Distance threshold *t*
  - Choose *t* so probability for inlier is *p* (e.g. 0.95)
  - Zero-mean Gaussian noise with std. dev. σ: $t^2=3.84\sigma^2$
- Number of samples *N*
  - Choose *N* so that, with probability *p*, at least one random sample is free from outliers (e.g. *p*=0.99) (outlier ratio: *e*)
- Consensus set size *d*
  - Should match expected inlier ratio

Source: M. Pollefeys

Adaptively determining the number of samples

- Inlier ratio *e* is often unknown a priori, so pick worst case, e.g. 50%, and adapt if more inliers are found, e.g. 80% would yield *e*=0.2
- Adaptive procedure:
  - *N*=∞, *sample_count* =0
  - While *N* >*sample_count*
    - Choose a sample and count the number of inliers
    - Set e = 1 – (number of inliers)/(total number of points)
    - Recompute *N* from *e:*

$$N = \log(1-p)/\log\left(1-(1-e)^s\right)$$

    - Increment the *sample_count* by 1

Source: M. Pollefeys

RANSAC pros and cons

- Pros
  - Simple and general
  - Applicable to many different problems
  - Often works well in practice
- Cons
  - Lots of parameters to tune
  - Can’t always get a good initialization of the model based on the minimum number of samples
  - Sometimes too many iterations are required
  - Can fail for extremely low inlier ratios
  - We can often do better than brute-force sampling

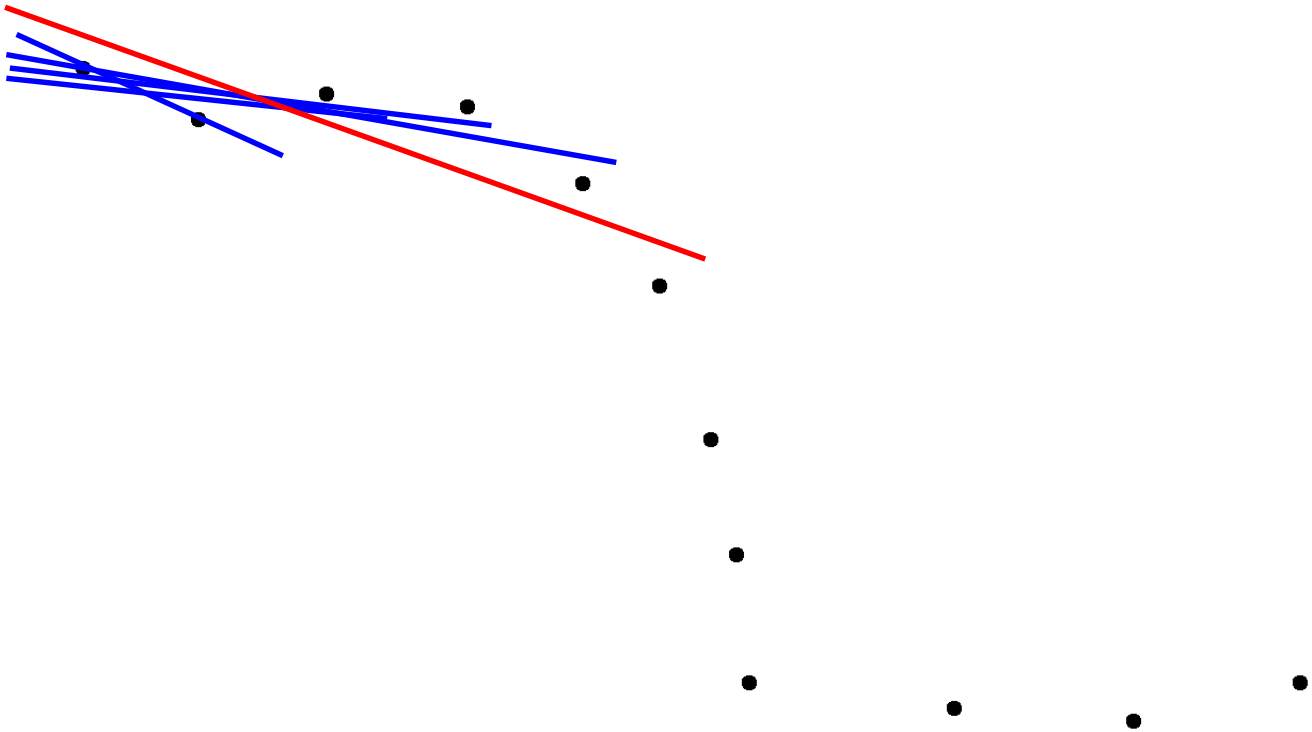
Fitting multiple lines

- Voting strategies
  - Hough transform
  - RANSAC
- Other approaches
  - Incremental line fitting
  - K-lines
  - Expectation maximization

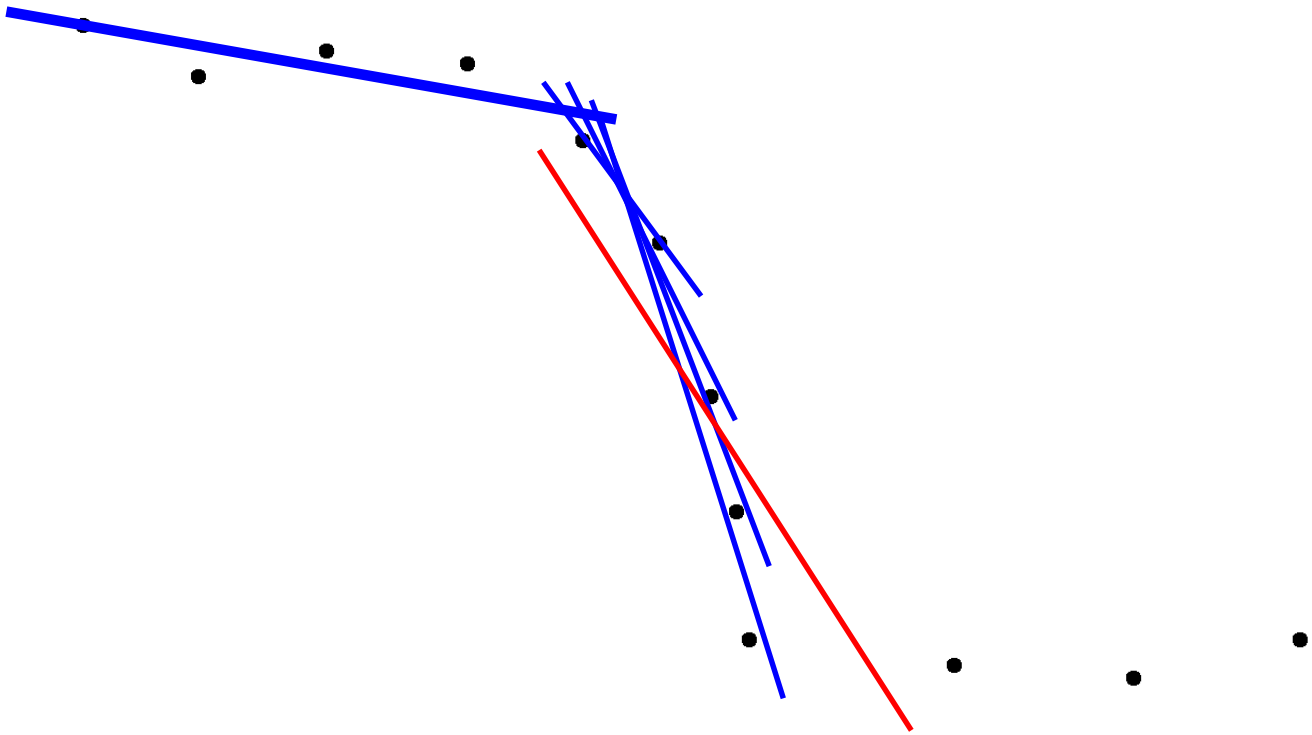
Incremental line fitting

- Examine edge points in their order along an edge chain
- Fit line to the first *s* points
- While line fitting residual is small enough, continue adding points to the current line and refitting
- When residual exceeds a threshold, break off current line and start a new one with the next *s* “unassigned” points

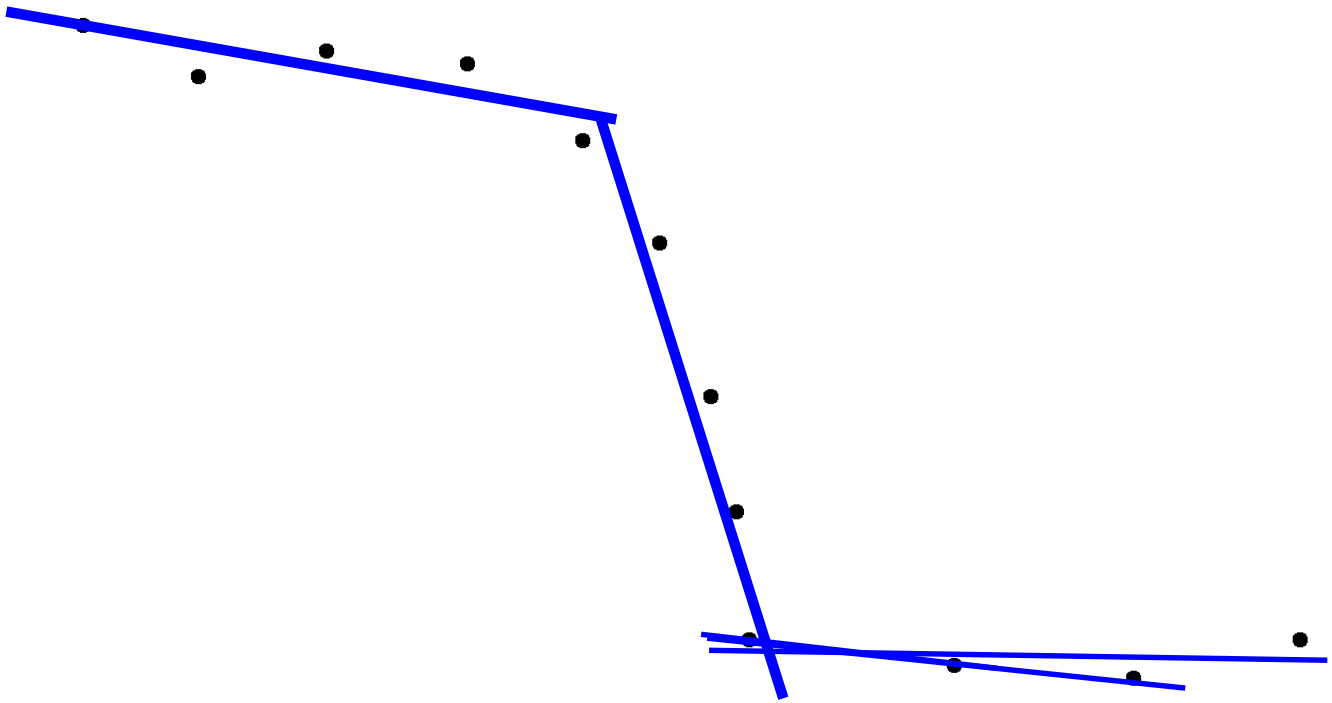
Incremental line fitting



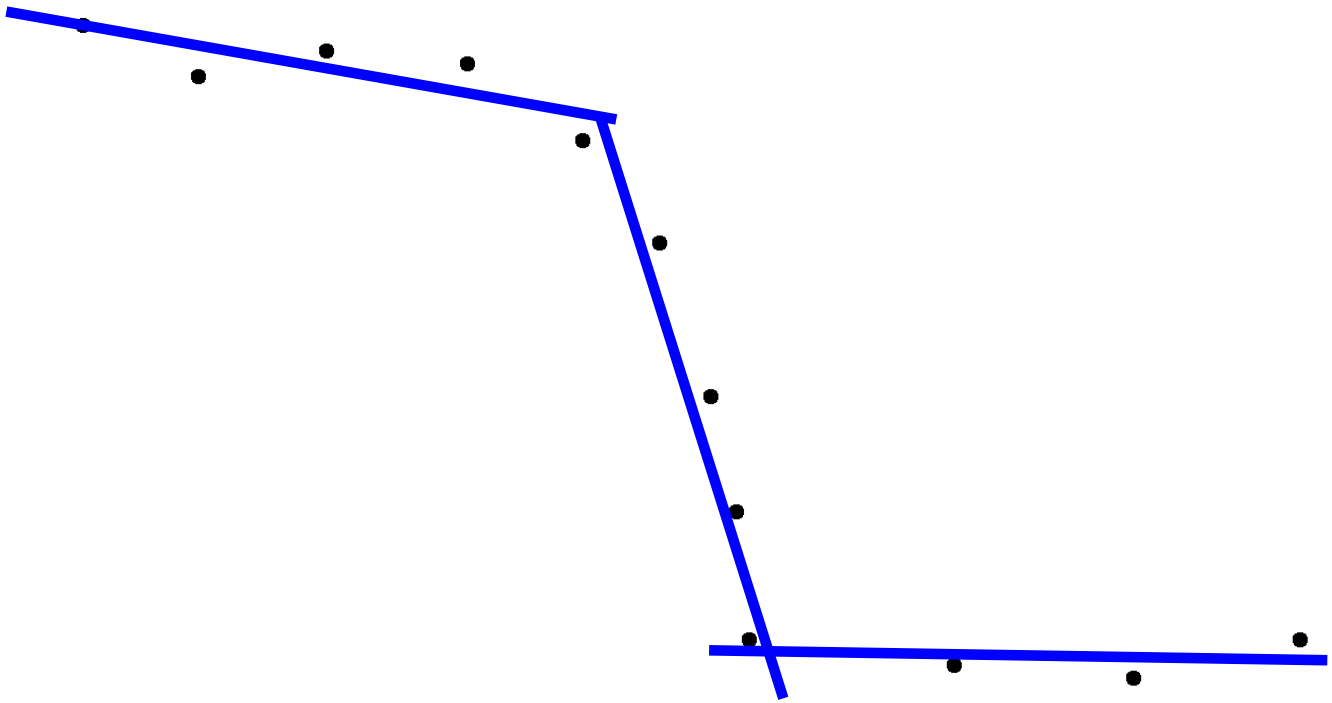
Incremental line fitting



Incremental line fitting



Incremental line fitting



Incremental fitting pros and cons

- Pros
  - Exploits locality
  - Adaptively determines the number of lines
- Cons
  - Needs sequential ordering of features
  - Can’t cope with occlusion
  - Sensitive to noise and choice of threshold

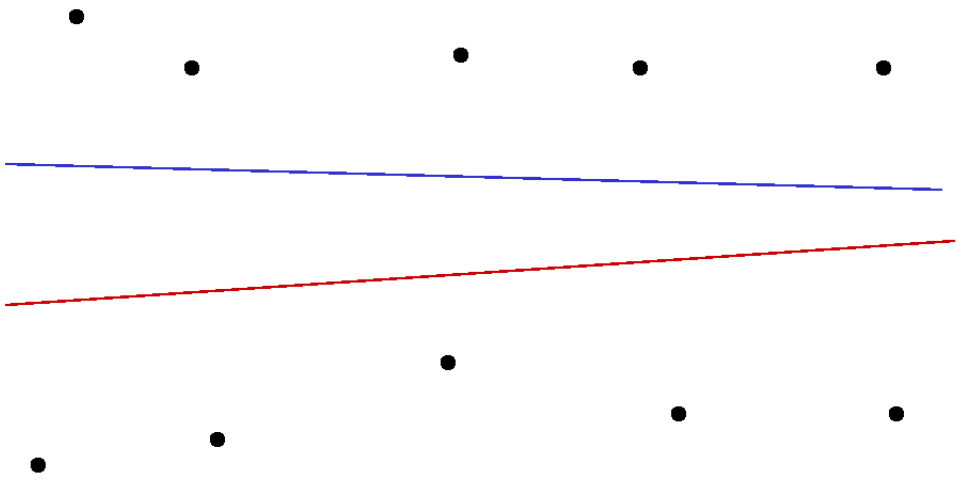
Last time: Fitting

- Fitting without outliers: least squares
  - Probabilistic interpretation
  - Least squares for general curves
- Dealing with outliers
  - Robust fitting
  - RANSAC
- Fitting multiple lines
  - Voting methods: Hough transform, RANSAC
  - Incremental line fitting
  - Next: K-lines

K-Lines

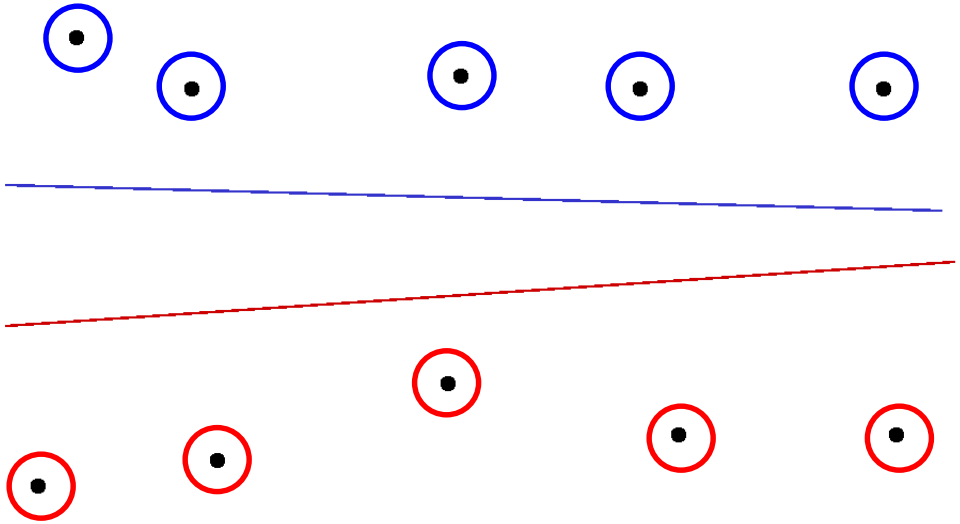
- Initialize *k* lines
  - Option 1: Randomly initialize *k* sets of parameters
  - Option 2: Randomly partition points into *k* sets and fit lines to them
- Iterate until convergence:
  - Assign each point to the nearest line
  - Refit parameters for each line

K-Lines example 1



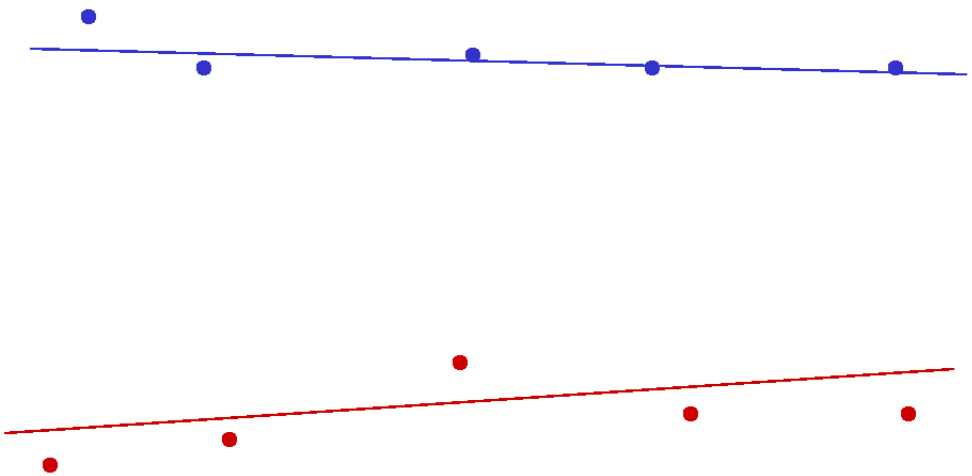
initialization

K-Lines example 1



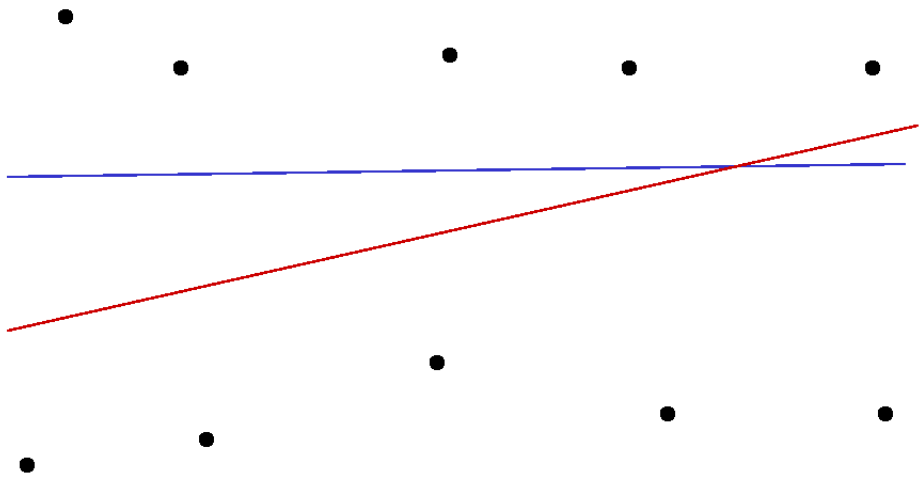
iteration 1: assignment to nearest line

K-Lines example 1



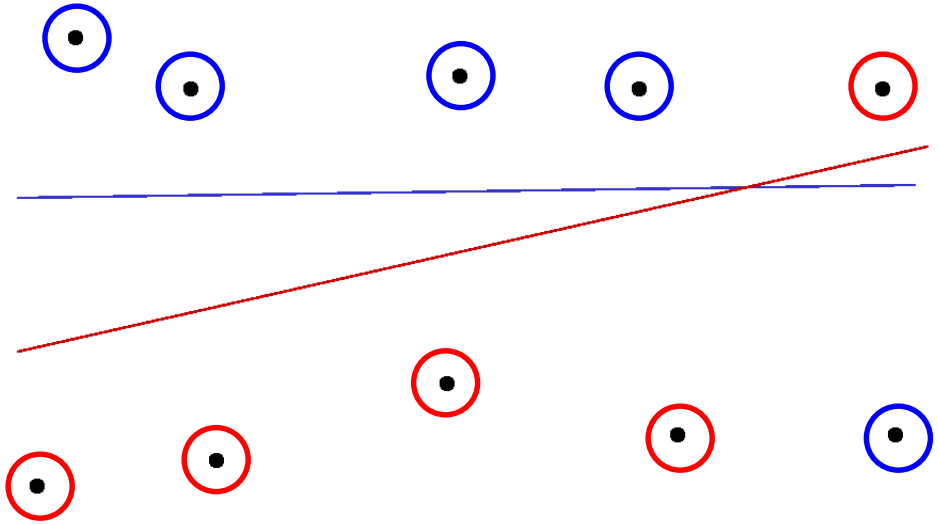
iteration 1: refitting

K-Lines example 2



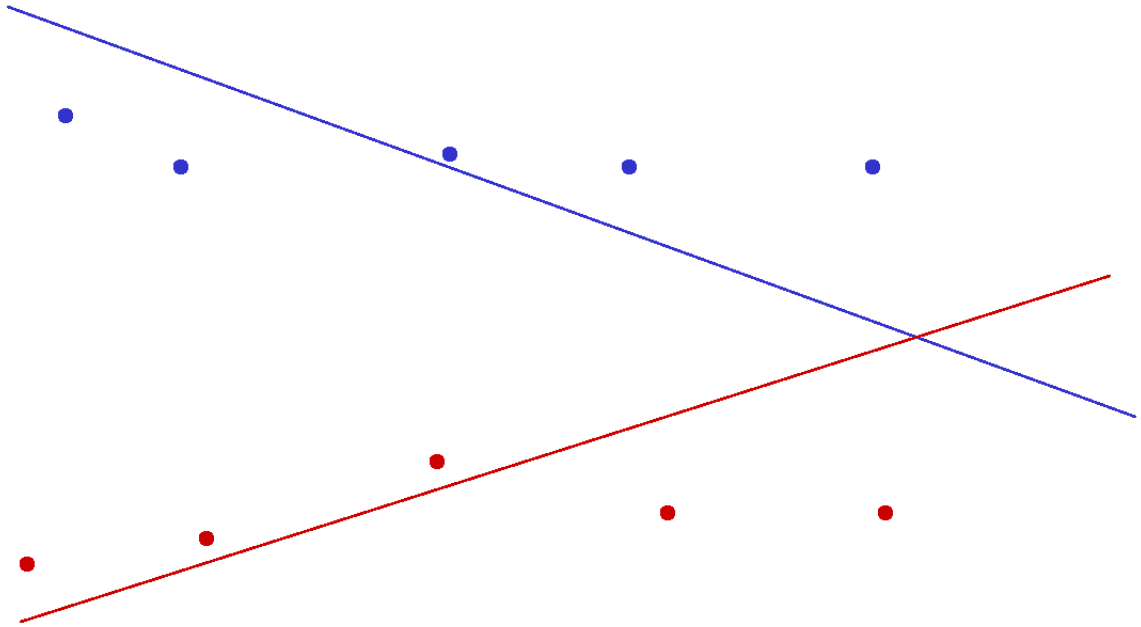
initialization

K-Lines example 2



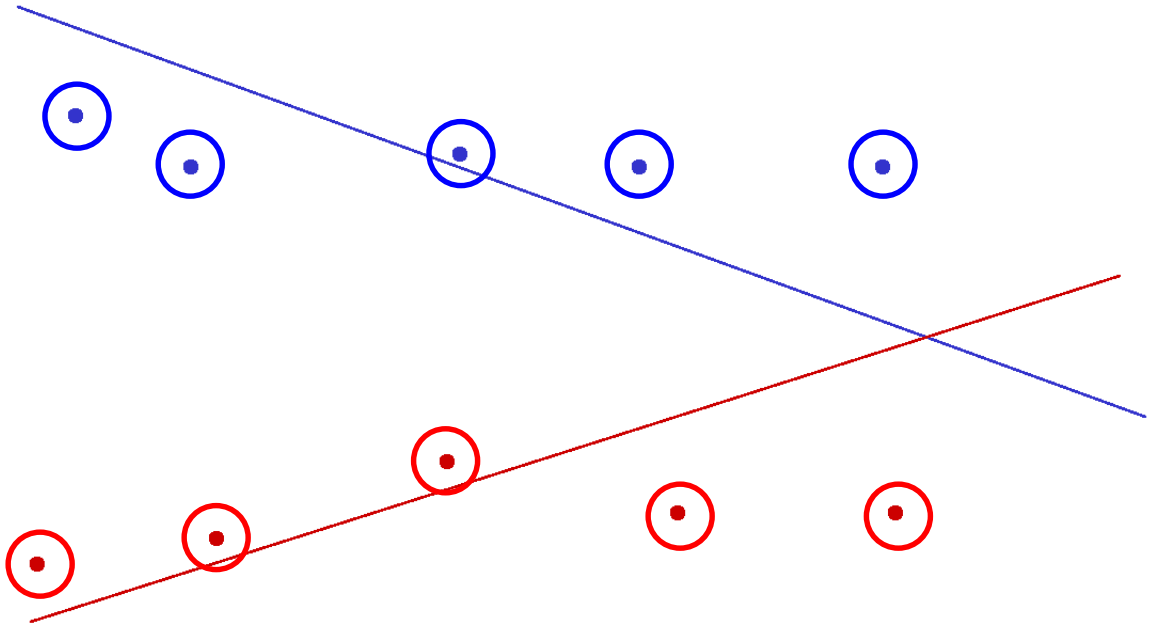
iteration 1: assignment to nearest line

K-Lines example 2



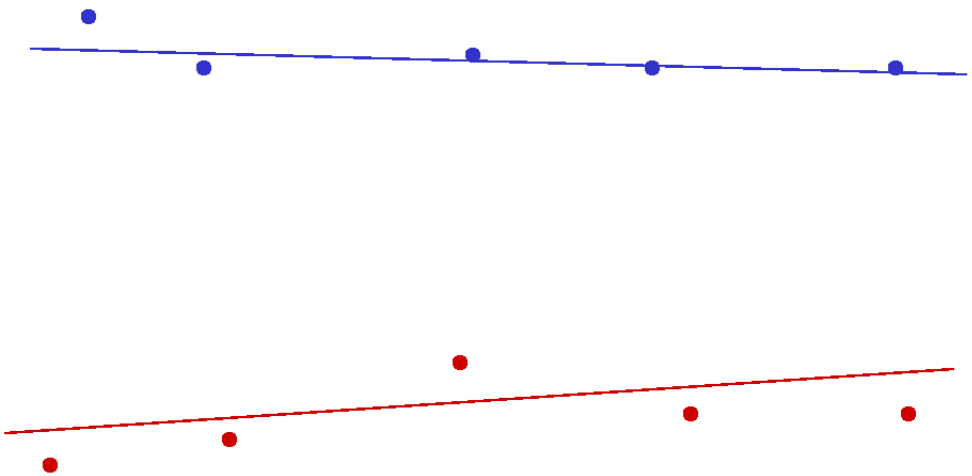
iteration 1: refitting

K-Lines example 2



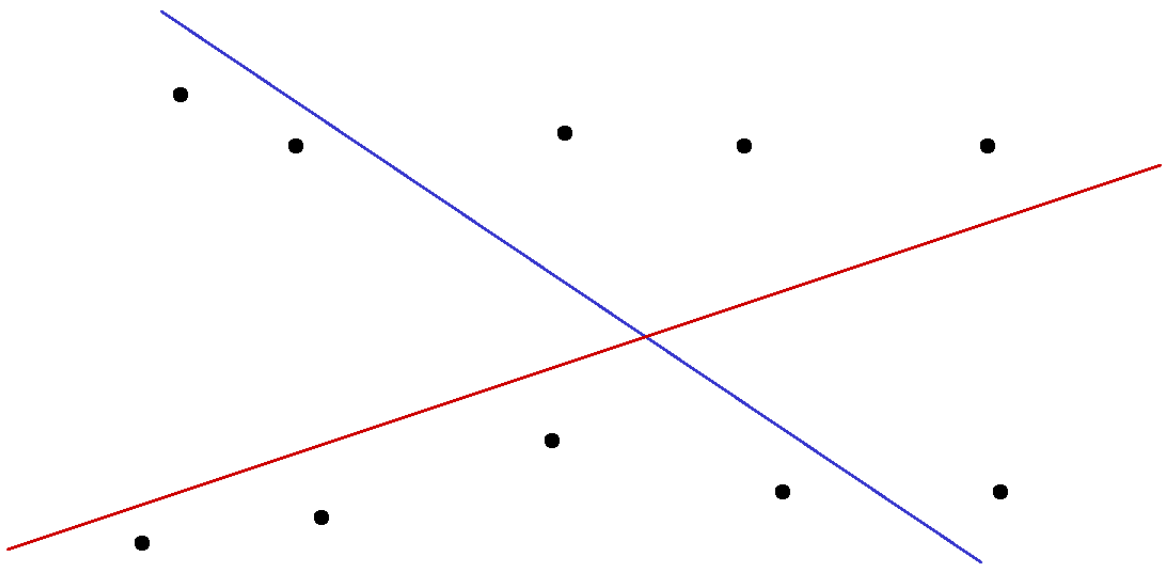
iteration 2: assignment to nearest line

K-Lines example 2



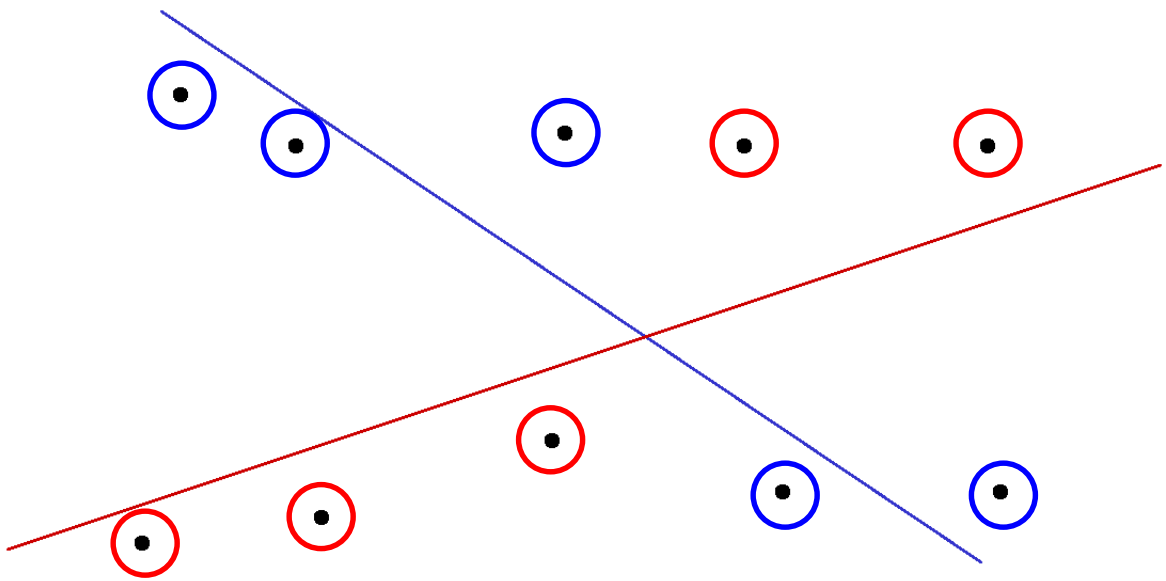
iteration 2: refitting

K-Lines example 3



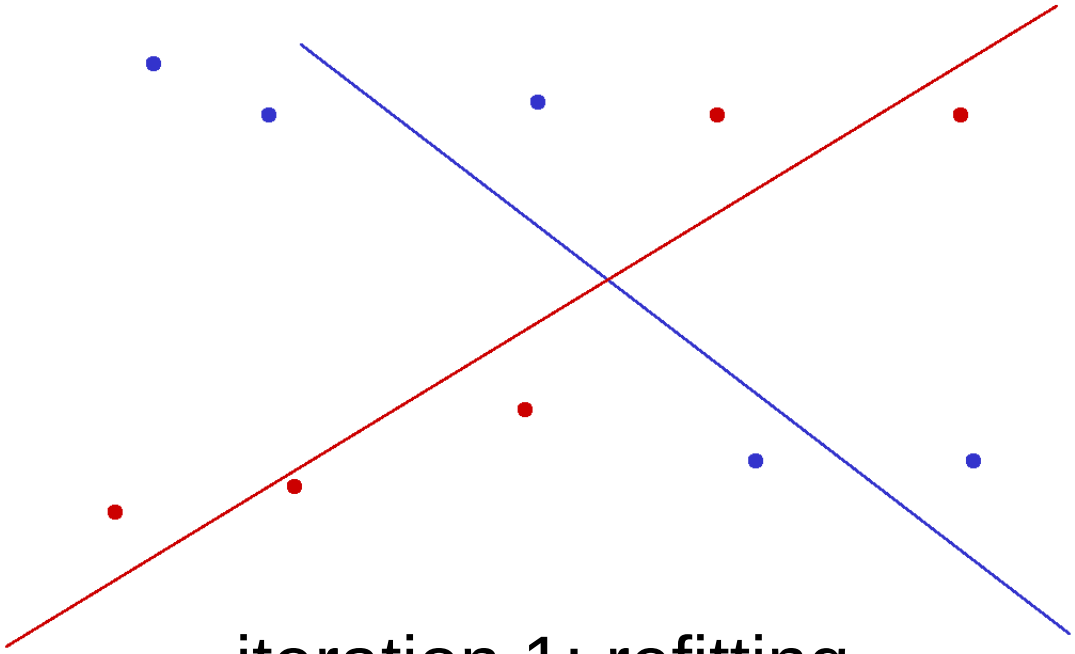
initialization

K-Lines example 3



iteration 2: assignment to nearest line

K-Lines example 3



iteration 1: refitting

K-Lines pros and cons

- Pros
  - Guaranteed to reduce line fitting residual at each iteration
  - Can cope with occlusion
- Cons
  - Need to know *k*
  - Can get stuck in local minima
  - Sensitive to initialization

Expectation Maximization (EM)

- A more general probabilistic method for maximizing likelihood in the presence of *missing data* (i.e., assignments of points to lines)

Expectation Maximization (EM)

- We want to maximize the likelihood of the data points given parameters $\theta = \{a_k, b_k, d_k\}$ of our lines:

$$-\log L(\theta) = \sum_k \sum_{i \in l_k} (a_k x_i + b_k y_i - d_k)^2$$

Expectation Maximization (EM)

- We want to maximize the likelihood of the data points given parameters $\theta = \{a_k, b_k, d_k\}$ of our lines:

$$-\log L(\theta) = \sum_k \sum_{i \in l_k} (a_k x_i + b_k y_i - d_k)^2$$

- Missing data: assignment of points i to lines l_k
- Iterate:
  - Guess missing data using current model parameters
  - Form maximum likelihood estimate of parameters using filled-in values for missing data
- Technical detail: need to take the expectation with respect to the distribution of the missing data
  - The resulting procedure is guaranteed to increase likelihood at every iteration
- More on this later in the course (hopefully)

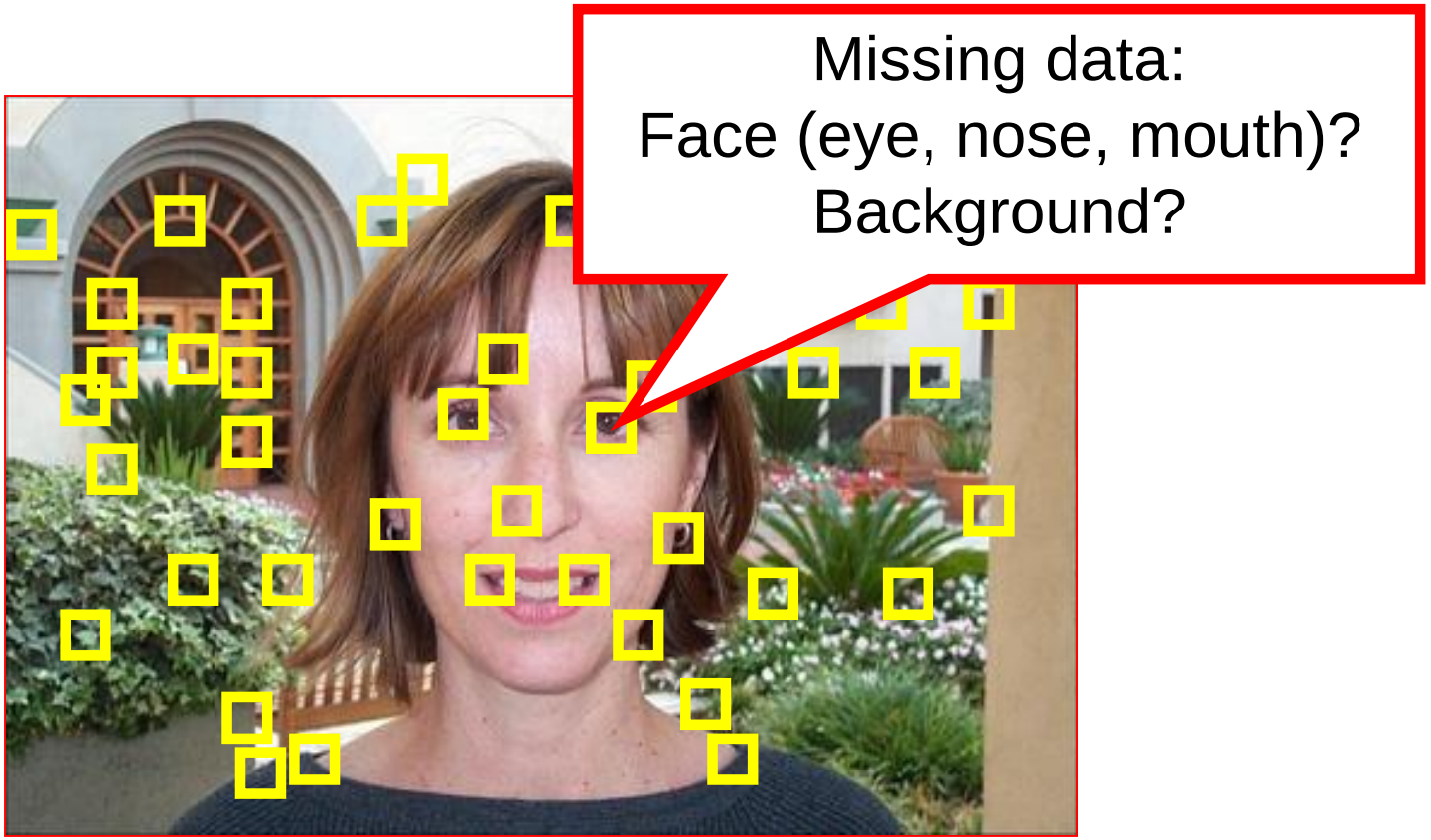
Application to recognition (teaser)

- Unsupervised learning: the target object is not segmented in the training image

M. Weber, M. Welling and P. Perona, Unsupervised learning of models for recognition, ECCV 2000

Application to recognition (teaser)

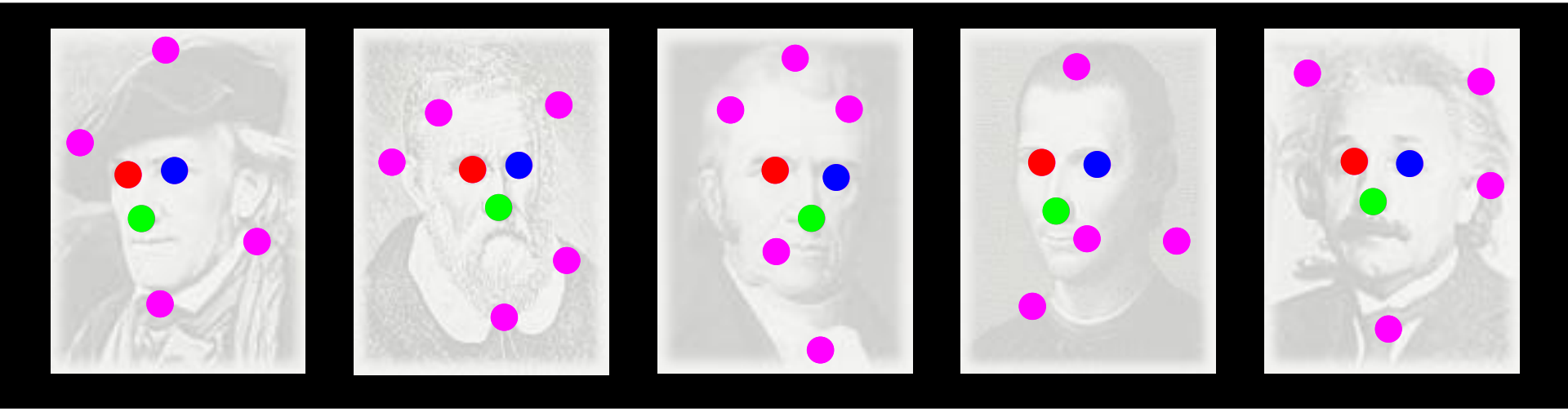
- Unsupervised learning: the target object is not segmented in the training image



M. Weber, M. Welling and P. Perona, Unsupervised learning of models for recognition, ECCV 2000

Learning using EM

- Task: Estimation of model parameters
- Chicken and Egg type problem, since we initially know neither:
  - Model parameters
  - Assignment of regions to parts
- Let the assignments be a hidden variable and use EM algorithm to learn them and the model parameters



Slide credit: R. Fergus

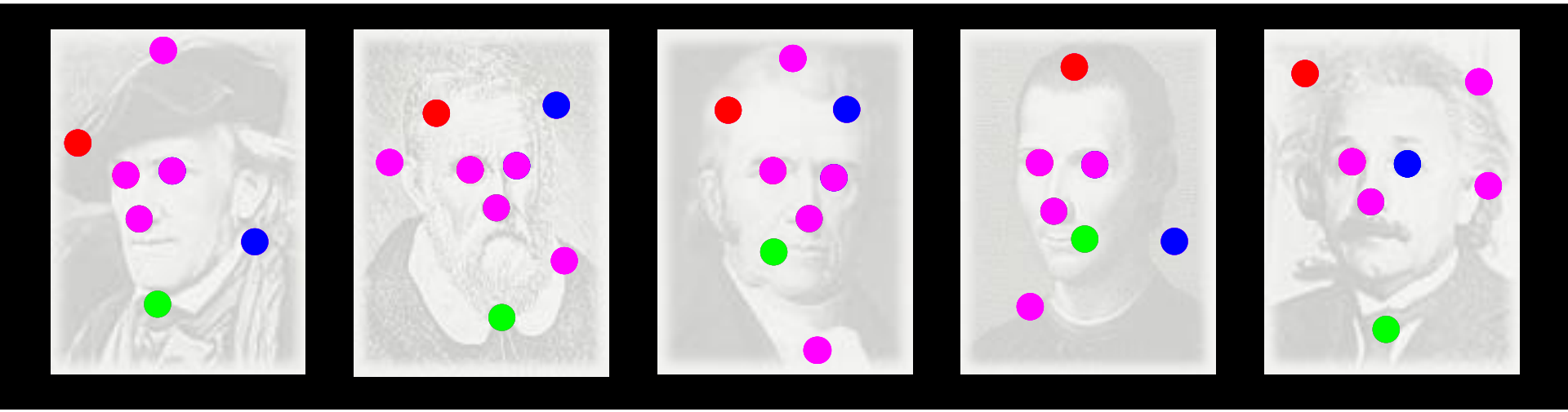
Learning procedure

- Find regions & their location & appearance
- Initialize model parameters
- Use EM and iterate to convergence:

  E-step: Compute assignments for which regions belong to which part

  M-step: Update model parameters

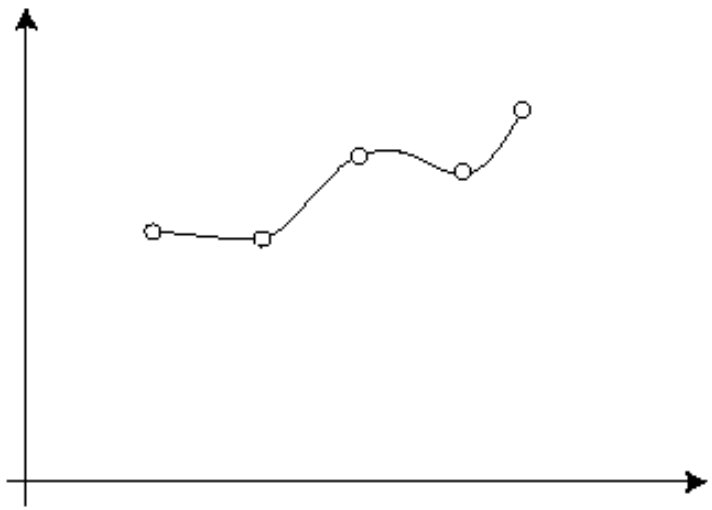
- Trying to maximize likelihood – consistency in shape & appearance

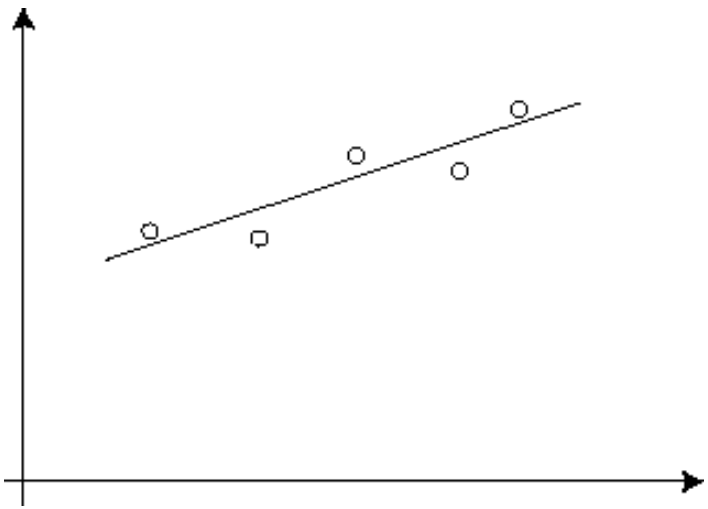


Slide credit: R. Fergus

Model selection

- Should we prefer a simpler or a more complex model?
- Two issues
  - Which model fits the observed data best?
  - Generalization performance: how well will a model predict points we haven't seen before?

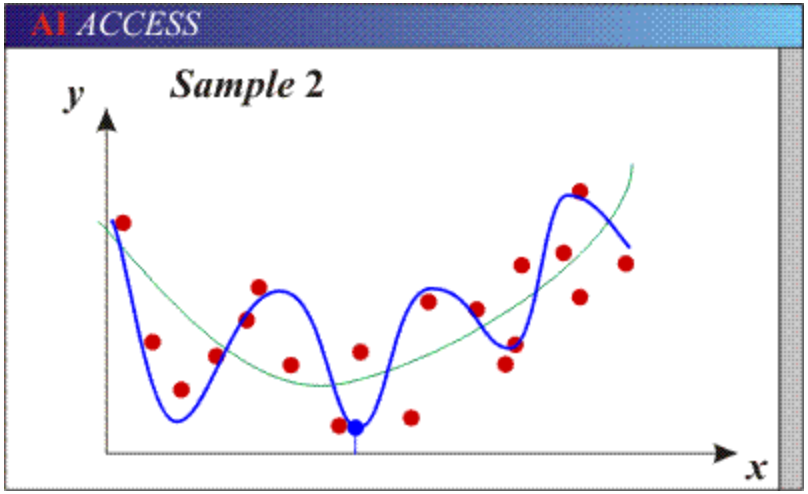


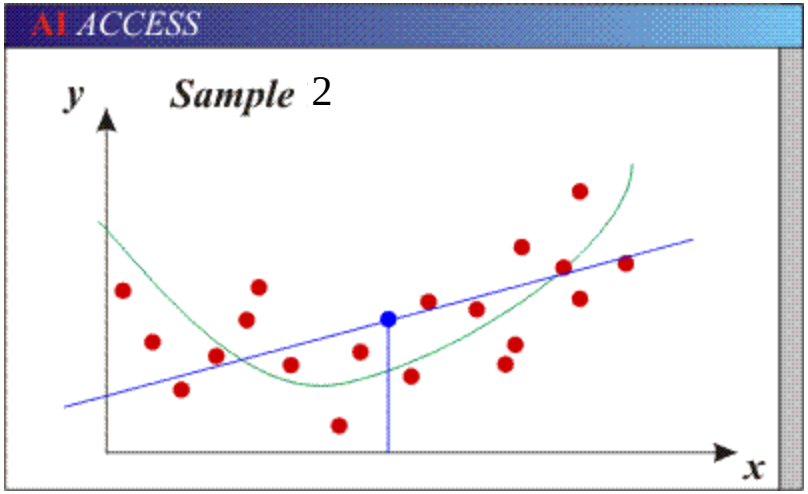


Bias-variance tradeoff

- Models with too many parameters may fit a given sample better, but have high *variance*
- Generalization error is due to *overfitting*

- Models with too few parameters may not fit a given sample well because of high *bias*
- Generalization error is due to *underfitting*





Occam's razor

- Given several models that describe the data equally well, the simpler one should be preferred
- There should be some tradeoff between error and model complexity
  - This is rarely done rigorously, but is a powerful "rule of thumb"
  - Simpler models are often preferred because of their robustness (= low variance)

Review of key concepts

- Least squares fitting
- Probabilistic fitting
  - The likelihood function
  - Estimates: maximum likelihood, MAP
- Dealing with outliers
  - Robust fitting
  - RANSAC
- Fitting multiple lines
  - Incremental fitting
  - Voting: Hough transform, RANSAC
  - Alternating minimization with “missing data”: K-lines, EM
- Model selection
  - Bias vs. variance
  - Occam’s razor